

Plan: HyMeYOLO — k -cycle structural attention on convolutional feature maps for object detection

HyMeKo / HSiKAN

2026-05-11

1 Scope and goal

HyMeYOLO inserts a **k -cycle structural attention layer** between a YOLO-style convolutional backbone and the detection head. The k -cycle layer operates on a signed graph induced by spatial feature similarities: vertices are spatial locations on a feature map, edges are local-neighbourhood connections, and signs are $\text{sign}(\langle f_i, f_j \rangle)$ for feature vectors f_i, f_j . Top- K balanced cycles enumerated on this signed graph contribute extra channels (cycle-density heatmaps) to the detection head's input.

Why now: pure HSiKAN-on-vision was a documented null result (`project_hsikan_vision_redo_2026_05_11`). HGNN-style on MNIST/Fashion lost to CNN at $h=16/n=1$). Vision regime needs translation equivariance, which CNN provides and HGNN-on-pixels lacks. **HyMeYOLO keeps the CNN backbone** and adds signed-cycle reasoning *on top of conv features*. Tonight's infrastructure (SoA cycle enum, vertex prefilter, FPN-tiered caps, dedup optimisations) brings the per-feature-map cycle cost from ~ 100 ms down to ~ 20 ms — fast enough to be a single GPU step.

Concrete goal: a feasibility experiment on Cluttered MNIST (synthetic, 5,000 images, 1–3 overlapping digits each) measures whether HyMeYOLO improves $\text{mAP}_{0.5}$ vs a matched vanilla YOLO-style baseline at the **same backbone, same train budget**. If feasibility passes ($\geq +0.01$ $\text{mAP}_{0.5}$, paired across 5 seeds), promote to PASCAL VOC 2007. Only if that passes, promote to COCO.

2 Affected files

- `hymeko_yolo/` — NEW. Rust+Python crate/module containing the cycle-attention layer, the synthetic data generator, and the YOLO-style detection head. Layout:

```
hymeko_yolo/
  Cargo.toml                (Rust crate, depends
  hymeko_graph + hymeko_py)
  pyproject.toml
  README.md
  src/
    lib.rs                  Rust ndarray helpers (sign,
  neighbourhood)
    cycle_features.rs       Signed-feature-graph builder
  (Rust)
    python/
      __init__.py
      synthetic.py          Cluttered MNIST generator
      backbone.py           ResNet-18 / DarkNet-Tiny
  wrappers
    cycle_layer.py          PyTorch module wrapping the
  Rust enum
```

	detection_head.py	YOLO-style anchor + class
head		
	train.py	Training loop, mAP eval
	viz.py	Cycle-heatmap overlays for
sanity check		
tests/		
	test_synthetic.py	synth data invariants
	test_signed_graph_from_feats.py	sign/neighbour
correctness		
	test_cycle_layer.py	forward/backward parity
	test_train_smoke.py	end-to-end 5-image smoke

- No changes to existing `hymeko_graph`, `hymeko.py`, or `signedkan_wip` — HyMeYOLO is a **consumer** of the cycle infrastructure, not a modifier.

3 CORE.YAML items touched

None. HyMeYOLO is a new module in its own directory.

4 Interface changes

Rust: signed feature graph builder

```
// hymeko_yolo/src/cycle_features.rs
//
// Build a SignedGraph from a (B, C, H, W) feature map. Per-spatial-
// position feature vector f_{h,w} R^C. Edges are 8-connected;
// signs are sign(<f_i, f_j>) of normalised feature vectors.
pub fn build_signed_feature_graph(
    feats: &[f32],           // shape (H*W, C) row-major after batch
    unbinding
    h: usize, w: usize, c: usize,
    neighbourhood: u8,       // 4 | 8 | 16 (k-NN ring)
    sign_threshold: f32,     // tau; |dot| < tau    drop edge
) -> SignedGraph;
```

Python: PyTorch cycle-attention layer

```
# hymeko_yolo/src/python/cycle_layer.py
class CycleAttention(nn.Module):
    """k-cycle structural attention on conv feature maps.

    Forward:
    x : (B, C, H, W)    conv feature map
    for each batch item:
    1. f_normalised = F.normalize(x.view(B,C,-1), dim=1)
    2. signed_graph = hymeko_yolo.build_signed_feature_graph(...)
    3. cycles = hymeko.
       enumerate_top_k_per_vertex_cycles_signed_filtered_rs(
           ..., k_len=self.k, m_per_vertex=self.m,
           'fraction_negative', 'balance',
           'compose:degree,triangle', 2,
       )
    4. cycle_heatmap : (B, k+1, H, W)
       - 1 channel = cycle-membership count per spatial position
```

```

        - k channels = score of cycles whose centroid lands here
                      stratified by canonical-rotation-position
    out : torch.cat([x, cycle_heatmap], dim=1)    (B, C+k+1, H, W)
    """
    def __init__(self, k=3, m=16, neighbourhood=8, sign_threshold=0.1,
                  detach_graph=True):
        ...
    def forward(self, x): ...

```

Differentiability caveat: the cycle enumeration itself is *not* differentiable w.r.t. feature values. The `detach_graph` flag (default `True`) means gradients do not flow through the graph-building step; the cycle-heatmap is treated as an extra feature. This is the v1 approach (**cycle features as auxiliary signal**, not learned via end-to-end attention). A future v2 could implement differentiable subgraph selection via straight-through-estimator on the edge-sign threshold.

Python: detection head

```

# hymeko_yolo/src/python/detection_head.py
class YOLOHead(nn.Module):
    """Standard YOLO-style head: anchors    (4 box + 1 conf + n_cls).
    Accepts (B, C', H, W) input where C' = backbone_C + cycle_channels.
    """

```

5 Test strategy (CLAUDE.md §3)

Unit (Rust)

- `build_signed_feature_graph`: 3×3 hand-coded feature map $\rightarrow$ expected edge set + signs.
- Sign threshold $\tau = 0$ produces all edges with ± 1 signs; $\tau \rightarrow \infty$ produces no edges.
- 4-, 8-, 16-neighbourhood produce expected edge counts.

Unit (Python)

- `CycleAttention` forward on a fixed seed produces bit-deterministic output.
- Cycle heatmap shape $(B, k+1, H, W)$ and dtype `torch.float32`.
- Cluttered MNIST generator: 2 digits / image, 1 digit / image, and 3 digits / image cases all produce valid bbox annotations (no negative widths/heights, all in-image).
- Backward pass on the detection head only (graph detached) produces non-zero gradients for backbone + head.

Integration

- End-to-end 5-image smoke: 1 epoch on 5 Cluttered MNIST images, check loss decreases.
- mAP eval on a held-out 10-image fixture: model output structure matches detection-eval expectations.

Performance test

- `CycleAttention.forward(x)` on a $(2, 64, 32, 32)$ input: median wall ≤ 30 ms (5-iter post-warmup, RTX 2070 SUPER).
- Memory: forward-pass extra RSS ≤ 200 MB per image (cycle storage + heatmap).
- End-to-end Cluttered MNIST train epoch: ≤ 60 s (200 batches $\times$ 32 images).

6 Performance budget

- Feasibility wall: ≤ 60 min total for 50-epoch training on Cluttered MNIST (RTX 2070 SUPER, batch=32).
- Memory: ≤ 6 GB GPU, ≤ 8 GB CPU.
- $\text{mAP}_{0.5}$ vs vanilla baseline: $\geq +0.01$ paired 5-seed.
- If feasibility passes: PASCAL VOC training budget ≤ 6 h for full 50-epoch run.

7 Risk anticipation (CLAUDE.md §2)

Performance-contract preservation

The cycle layer consumes `hymeko`’s `PyO3` bindings as-is. The post-SoA-refactor wall budget (`enumerate_top_k_per_vertex_cycles_signed_filtered_rs` at ~ 100 ms on a 1k-vertex graph is the contract we depend on). If this regresses, HyMeYOLO’s per-batch cost blows up proportionally. Mitigation: pin a `bench_cycle_layer.py` that asserts the 30 ms forward-pass budget; add to CI.

Aspirational comments to honour

The HSiKAN-vision null (`project_hsikan_vision_redo_2026_05_06`) says “vision regime favours translation equivariance over signed-cycle structure”. HyMeYOLO sidesteps this by keeping the CNN backbone; the cycle layer is *additive*, not replacing the backbone. If the feasibility experiment fails (no mAP improvement), the conclusion is consistent with the prior null: signed-cycle structure alone is not enough for vision tasks. We then *do not* promote to PASCAL.

Worst-case input

Cluttered MNIST: 32×32 feature map (so 1024 vertices), batch size 32, $k = 3$, $m = 16$. Total cycles enumerated per batch $\approx 32 \times 16,000 = 512,000$ cycle ids. Subsampled to $\leq 1,000$ cycles per image before feeding the heatmap; this keeps the post-enum memory bounded.

What could go wrong

- **Cycle features are constant-ish across positions:** if every position participates in many cycles, the cycle-density heatmap looks like uniform noise and the detection head can’t use it. Mitigation: visualise the heatmap on a couple of images early; if it’s flat, the sign threshold or neighbourhood is wrong, not the architecture.
- **Sign threshold τ is the wrong knob:** cosine-sim sign depends on normalisation; might need learnable scale. Add as a v2 path if v1 fails.
- **Synthetic $\rightarrow$ real gap:** Cluttered MNIST may not transfer to PASCAL. Mitigation: the plan has a halt gate after each dataset stage; we don’t promote to the next dataset unless the previous one shows signal.
- **Backbone choice biases the result:** ResNet-18 is a reasonable default but may interact poorly with cycle features. Mitigation: also try DarkNet-Tiny.

8 Halt conditions

- Cluttered MNIST feasibility: HyMeYOLO $\text{mAP}_{0.5} < \text{vanilla baseline} + 0.005$ (paired 5-seed) $\rightarrow$ halt, document as null result.
- Cycle layer wall > 50 ms on a 32×32 feature map $\rightarrow$ performance contract violated, halt and investigate.

- PASCAL VOC $mAP_{0.5}$ regresses vs the matched baseline $\rightarrow$ halt, do not promote to COCO.

9 Implementation order

Phase 0 (this plan, $\sim 3-5$ days)

1. Rust: `build_signed_feature_graph` + PyO3 binding exposing it as `hymeko.build_signed_feature_graph`.
2. Python: `synthetic.py` Cluttered MNIST generator + 8 unit tests.
3. Python: `cycle_layer.py` `CycleAttention` forward (graph detached); unit tests for shape/determinism.
4. Python: `detection_head.py` YOLO-style head; forward + backward unit tests.
5. Python: `train.py` 5-image smoke test; loss decreases in 1 epoch.
6. Python: `viz.py` qualitative visualisation; manual inspection of cycle-density heatmaps.

Phase 1 (gated on phase 0 passing, ~ 2 days)

1. Full Cluttered MNIST training: HyMeYOLO vs vanilla baseline, 5-seed paired, 50 epochs each. Wall budget ≤ 60 min total.
2. Sweep cycle layer hyperparameters ($k \in \{3, 4\}$, $m \in \{8, 16, 32\}$, $\tau \in \{0, 0.1, 0.2\}$).
3. Report: mAP, wall time, peak memory, cycle-heatmap stats.

Phase 2 (gated on Phase 1, ~ 1 week)

1. PASCAL VOC 2007 training, 50 epochs, ResNet-18 backbone.
2. Compare to YOLOv5n baseline trained on the same data with the same backbone. Report $mAP_{0.5}$, $mAP_{0.5:0.95}$, per-class AP.

Phase 3 (gated on Phase 2)

COCO subset, then full COCO.

10 Rollback path

HyMeYOLO is a new directory; nothing depends on it. Reverting removes `hymeko_yolo/` cleanly.

11 Empty-plan-dir hygiene

This plan dir is fully populated (4 artefacts). If abandoned, the dir gets a `HALTED.md` with the abort reason.